Name:- Swayam
SAP ID:- 590021784
Batch:- 17

DSA ASSIGNMENT

Date / /

Q1:- Importance of Data Structures (Array Vs Unstructured Variable)

•) Without Data Structure (Unstructure Variables)

Eg:- storing 5 marks

int m1 = 78, m2 = 85, m3 = 90, m4 = 67, m5 = 88;

Problems:

i.) Cannot use loops easily
ii.) Hard to scale (100 students → 100 variables)
iii.) Searching requires manual checks
iv.) No indexing or grouping

Eg:- find averages

avg = (m1 + m2 + m3 + m4 + m5) / 5;

•) With Array (Organized Data Structure)

int m[5] = {78, 85, 90, 67, 88};

Advantages:

i) Indexed access
ii) Loop processing
iii) Easy searching & sorting
iv) Memory contiguous → fast

Date / /

Eg: Average:

```
sum = 0;
for (i=0; i<5; i++)
    sum += m[i];
avg = sum/5;
```

•) Efficiency comparison

Factor	Unstructure Variables	Array
Memory organization	Scattered	Contiguous
Scalability	Poor	Excellent
Processing	Manual	Loop-based
Searching	O(n) manual checks	O(n) loop
Sorting	Hard	Standard algorithms
Maintenance	Difficult	Easy

(CONCLUSION)

Data structures like arrays improve:

i.) efficiency
ii.) Scalability
iii.) ~~read~~ readability
iv.) algorithm applicability.

Date / /

Q2) Recursive Sum of Array

•) Idea

Sum of array = first element + sum of remaining array

Sum (n) = arr [0] + Sum (n-1)

Base case :

~~Sum~~ Sum (0) = 0

•) C Function

```
int sumArray (int arr [], int n)
{
    if (n == 0)
        return 0;

    return arr [n-1] + sumArray (arr, n-1);
```

•) ~~Example Trace~~

~~Array = {2, 4, 6}~~

~~Call Stack~~

Date / /

•) Example Trace

Array = {2, 4, 6}

Call stack :

SumArray (arr, 3)
= arr[2] + SumArray (arr, 2)
= 6 + SumArray (arr, 2)

SumArray (arr, 2)
= arr[1] + SumArray (arr, 1)
= 4 + ~~Sum~~ SumArray (arr, 1)

SumArray (arr, 1)
= arr[0] + SumArray (arr, 0)
= 2 + 0

Return :-
SumArray (1) = 2
SumArray (2) = 4 + 2 = 6
SumArray (3) = 6 + 6 = 12

(Final sum = 12)

Q3. Singly Linked List Design & Operations.

•) Node Structure

~~Stru~~ struct node
{
char ~~ch~~ data;
struct ~~no~~ node * next;
};

Date / /

•) ~~Ini~~ Initial Insert : HAPPY

List :

H → A → P → P → Y → ~~Null~~ NULL

i•) Insert Operations

Insert B at 3rd position

H → A → B → P → P → Y

ii) Insert T at 5th position

H → A → B → P → T → P → Y

iii) Insert at head I

I → H → A → B → P → T → P → Y

iv) Insert at tail U

I → H → A → B → P → T → P → Y → U

•) Search Operations

Search B → Found at positions 4
Search T → Found at position 6
Search Y → Found

Date / /

•) Delete Operations

Delete B:

I → M → A → P → T → P → Y → U

~~Del~~ Delete 3rd position (A):

I → M → P → T → P → Y → U

•) Update : Replace all P → W

I → M → W → T → W → Y → U

•) Count number of A

A = 0

•) Palindrome check

List : I M W T W Y U
Reverse : U Y W T W M I

~~Not~~ (Not palindrome)

Date / /

Q4:- Doubly Linked List

* Introduction

→ A doubly linked list is similar to singly linked list, but each node contains two pointers:

1.) Pointer to previous node.
2.) Pointer to next node.

→ This allows traversal in both directions

* Structure

```
struct Node{
    char data;
    Node* prev;
    Node* next;
};
```

* Initial list

NULL ← H ⇄ A ⇄ P ⇄ P ⇄ Y → NULL

* After All Insertions →

NULL ← I ⇄ M ⇄ A ⇄ B ⇄ P ⇄ T ⇄ R ⇄ P ⇄ Y ⇄ U → NULL

* After Deletion and Update →

NULL ← I ⇄ H ⇄ W ⇄ T ⇄ W ⇄ Y ⇄ U → NULL

Date / /

* Advantages →

→ Can Transverse forward and backward
→ Easier deletion
→ Reverse traversal possible.

* Disadvantages

→ Extra memory required
→ M or complex pointer handling
→ Slightly slower than singly linked list.

* Check Palindrome

Step :-
1.) Set two pointers : start and end
2.) Compare data
3.) Move start forward and end backward.
4.) If mismatch → Not Palindrome

Result : Not Palindrome

Q5:- * Introduction
→ Both singly and doubly linked lists are dynamic linear data structure used to store data non-contiguously. The major difference lies in number of pointer and traversal capability.

Date / /

* Structural Difference

Feature	Singly Linked List	Doubly Linked List
Node Structure	→ Data + Next	→ Data + Prev + Next
Memory Usage	→ Less	→ More
Pointer count	→ 1	→ 2
Traversal	→ Forward only	→ Forward and Backward
Reverse Traversal	→ Not possible	→ Possible
Implementation	→ Simple	→ Complex

* Time Complexity Comparison

Operation	Singly LL	Doubly LL
Insert at Head	→ O(1)	→ O(1)
Insert at Tail	→ O(n)	→ O(n)
Insert ~~at~~ at Middle	→ O(n)	→ O(n)
Delete	→ O(n)	→ O(n)
Search	→ O(n)	→ O(n)
Reverse Traversal	→ Not possible	→ O(n)

* Conclusion

→ Singly Linked list is simple and memory efficient, while ~~clearly~~ doubly linked list provides more flexibility and ~~the~~ bidirectional travel at the cost of extra memory. Choice of structure depends on application ~~requirements~~ requirements.